Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение высшего образования «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

по дисциплине «Кроссплатформенная разработка»
студента Колохматова Андрея Максимовича группы ПИН-252т

Пояснительная записка
Шифр работы От-43-ПИН-252т-3 ЛР
Специальность 09.03.03 Программная инженерия

Преподаватель М.А. Мищенко
Студент А.М. Колохматов

Омск 2026

РЕФЕРАТ

Объектом выполненной работы является кроссплатформенное приложение «Рэп-каталог», разработанное на Flutter и языке Dart.

Целью данной работы является замена моковых данных каркаса из второй лабораторной работы данными из сети, из того же источника, что и в первой работе, — открытого программного интерфейса MusicBrainz, с показом загрузки и ошибки на экранах.

В работе применены HTTP-клиент dio, генерация DTO библиотекой json_serializable, цепочка слоя данных «API-клиент — источник данных — репозиторий», закрытая иерархия ошибок LoadException, постраничная загрузка по параметрам limit и offset, параллельная загрузка связанных записей через Future.wait и загрузка фотографий исполнителей из Викиданных запросом SPARQL.

Степень внедрения — для учебных целей.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 ПОСТАНОВКА ЗАДАЧИ 6
2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ 8
3 РЕАЛИЗАЦИЯ 10
3.1 Клиент Dio 10
3.2 DTO и генерация 10
3.3 API-клиенты 11
3.4 Источник данных и ошибки 11
3.5 Маппер и репозиторий 12
3.6 Фотографии исполнителей 13
3.7 Состояния и Cubit 13
3.8 Экраны 14
4 ПРОВЕРКА РАБОТЫ 15
ЗАКЛЮЧЕНИЕ 23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 24

ВВЕДЕНИЕ

Почти любое мобильное или веб-приложение получает данные с сервера. В отличие от моков, сеть отвечает не сразу и не всегда: запрос может идти секунды, сервер может быть недоступен, может ответить ошибкой или ограничить частоту запросов. Поэтому переход с моков на сеть затрагивает не только слой данных: у каждого экрана появляются состояния загрузки и ошибки, а пользователь должен видеть, что произошло, и иметь возможность повторить действие.

Работа продолжает вторую лабораторную работу, в которой на Flutter был построен каркас каталога хип-хоп исполнителей на моковых данных. Моки были выгружены из MusicBrainz, и модель предметной области повторяла форму его ответов, поэтому замена источника должна затронуть папку data и сборку зависимостей в корне приложения, а экраны и Cubit — только в части загрузки и ошибки.

Цель работы — заменить моки данными из сети так, чтобы приложение честно показывало загрузку и ошибку, а слои не знали лишнего друг о друге.

Для достижения цели поставлены следующие задачи:

— подключить dio, json_annotation, build_runner и json_serializable;

— создать один экземпляр Dio с базовым адресом, таймаутами и журналом запросов в режиме отладки и передать его в API-клиенты через конструктор;

— описать DTO ответов сервера с генерацией fromJson и маппер DTO в доменную модель;

— реализовать источник данных, который переводит DioException в собственную ошибку LoadException, и репозиторий, который берёт данные из источника;

— добавить в Cubit списка и детали состояния загрузки и ошибки, а на экраны — сообщение и кнопку «Повторить»;

— сохранить поведение поиска из второй работы, включая сообщение «Ничего не найдено»;

— реализовать постраничную подгрузку списка без повторов и параллельную загрузку связанных записей на детали;

— показывать фотографии всех исполнителей каталога, а не только 33 записей из моков;

— удалить моки из каталога lib и проверить работу приложения в браузере, в том числе без сети и при ошибке сервера.

1 ПОСТАНОВКА ЗАДАЧИ

Источник данных — программный интерфейс MusicBrainz по адресу https://musicbrainz.org/ws/2/, тот же, что в первой работе. Он не требует ключа, отдаёт JSON по параметру fmt=json, поддерживает поиск на языке запросов Lucene и постраничную выдачу параметрами limit и offset, а в ответе поиска сообщает общее число найденных записей count. Сервер отдаёт заголовок Access-Control-Allow-Origin: *, поэтому к нему можно обращаться из браузера.

У источника есть два ограничения. Первое — не более одного запроса в секунду с одного адреса; при превышении сервер отвечает кодом 503. Второе — сервер просит указывать заголовок User-Agent с контактами приложения, но в браузере этот заголовок задать нельзя, его подставляет сам браузер.

Короткая справка об исполнителе на русском и английском языках в MusicBrainz отсутствует: как и в предыдущих работах, она берётся из первого абзаца статей Википедии. Связь с Википедией идёт через Викиданные: у исполнителя в MusicBrainz есть ссылка на элемент Викиданных, а у элемента — названия статей в английской и русской Википедии.

Фотографий MusicBrainz тоже не отдаёт. Во второй работе снимки 33 исполнителей хранились в ресурсах приложения, но теперь каталог содержит тысячи записей, и для остальных фото берётся из Викиданных: у элемента исполнителя есть свойство P434 (идентификатор MusicBrainz) и свойство P18 (изображение на Викискладе).

Требования к результату приведены в таблице 1.

Таблица 1 — Требования к работе приложения

№	Действие	Ожидаемый результат
1	Открыть приложение	Индикатор загрузки, затем список из сети. Моков в lib нет
2	Открыть деталь	Деталь грузится своим запросом со своим индикатором
3	Открыть без сети или с неверным адресом сервера	Сообщение и кнопка «Повторить»; после восстановления сети нажатие возвращает список
4	Найти то, чего нет	«Ничего не найдено», как во второй работе
5	Долистать до конца списка	Подгружается следующая страница без повторов
6	Открыть деталь и сразу вернуться	В консоли нет ошибок
7	Открыть деталь со связанными записями	Связанные записи грузятся параллельно

Требования к устройству кода: цепочка «API-клиент — источник данных — репозиторий — Cubit — экран» обязательна; dio и DioException не встречаются в Cubit и виджетах; DTO не попадают в экран и Cubit; аннотации json_serializable не стоят в доменной модели; ошибка в слое данных не превращается в пустой список; базовый адрес сервера задан в одном месте.

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Структура по фичам из второй работы сохранена: фича artist разделена на слои data, domain и presentation, общий код лежит в common. Изменения сосредоточены в папке features/artist/data и в новой папке common/network; распределение обязанностей между слоями приведено в таблице 2.

Таблица 2 — Слои и их обязанности

Слой	Классы	Что делает
Сеть	DioClient, LoadException	Один экземпляр Dio: базовый адрес, таймауты, очередь запросов к MusicBrainz, повтор при 503, журнал в режиме отладки; закрытая иерархия ошибок загрузки
API-клиент	ArtistApi, WikiApi	Один метод на один адрес сервера, возвращает DTO
Источник данных	IArtistDataSource, NetworkArtistDataSource	Составляет запрос Lucene, вызывает API-клиент, ловит DioException и бросает LoadException
Маппер	ArtistDtoMapper	Переводит DTO в доменную модель ArtistModel
Маппер фото	WikidataImagesMapper	Строит прямые адреса миниатюр Викисклада
Репозиторий	ArtistRepository	Берёт DTO из источника, применяет маппер, собирает справку из Википедии и фото из Викиданных
Cubit	ArtistListCubit, ArtistDetailCubit	Кладёт в состояние загрузку, данные или ошибку; про dio и DTO не знает
Экран	ArtistListScreen, ArtistDetailScreen	Рисует состояние, вызывает методы Cubit

Зависимости собираются в корневом виджете App, там же, где во второй работе регистрировался репозиторий: DioClient.create() создаёт один экземпляр Dio, он передаётся в конструкторы ArtistApi и WikiApi, клиенты — в NetworkArtistDataSource, источник — в ArtistRepository, а репозиторий регистрируется в RepositoryProvider по интерфейсу IArtistRepository. Cubit по-прежнему получают интерфейс из контекста и не знают, откуда приходят данные.

Слой domain изменился минимально. Метод getArtists теперь принимает смещение offset и возвращает страницу ArtistPage — записи и общее число найденных на сервере. Список идентификаторов связанных

артистов заменён списком related с краткими записями: MusicBrainz отдаёт связи вместе с карточкой исполнителя, и отдельные запросы за каждым связанным артистом при ограничении в один запрос в секунду заняли бы несколько секунд. Метод getRelated поэтому стал не нужен.

3 РЕАЛИЗАЦИЯ

3.1 Клиент Dio

Класс DioClient — единственное место, где записан адрес MusicBrainz. Он создаёт Dio с таймаутом соединения 10 секунд и таймаутом получения ответа 15 секунд, а в режиме отладки добавляет LogInterceptor, который печатает адреса запросов и ответы в консоль.

Ограничение в один запрос в секунду соблюдается перехватчиком _ThrottleInterceptor. Запросы к хосту musicbrainz.org выстраиваются в очередь из Future и выпускаются по одному не чаще раза в 1,1 секунды; запросы к Викиданным и Википедии проходят без очереди. Второй перехватчик, _RetryOn503Interceptor, один раз повторяет запрос, на который MusicBrainz ответил 503, после паузы в 2 секунды, как рекомендует документация сервера. Если повтор тоже получает 503, ошибка уходит наверх и пользователь видит сообщение с кнопкой «Повторить».

Заголовок User-Agent с адресом репозитория задаётся только вне браузера: в вебе браузер запрещает его менять, и попытка привела бы к предупреждению в консоли.

3.2 DTO и генерация

DTO описаны в файлах dto/artist_dto.dart и dto/wiki_dto.dart: ArtistSearchDto (ответ поиска с полями count, offset, artists), ArtistDto и вложенные AreaDto, LifeSpanDto, TagDto, AliasDto, ReleaseGroupDto, RelationDto, UrlDto, а также DTO Викиданных и краткого содержания статьи Википедии. Ключи MusicBrainz записываются через дефис (sort-name, life-span, begin-area), поэтому классам задан параметр fieldRename: FieldRename.kebab, а генерация toJson отключена, так как приложение ничего не отправляет на сервер.

Поля, которые сервер может не прислать, объявлены необязательными, а списки имеют значение по умолчанию — пустой список. Решение о том, что показать вместо пустого поля, принимает маппер, а не DTO. Код fromJson генерируется командой dart run build_runner build --delete-conflicting-outputs, сгенерированные файлы *.g.dart хранятся в репозитории.

3.3 API-клиенты

ArtistApi содержит два метода по числу адресов: searchArtists выполняет GET artist с параметрами query, limit и offset, getArtist выполняет GET artist/{id} с параметром inc=aliases+tags+artist-rels+url-rels+release-groups, который за один запрос возвращает псевдонимы, теги, связи с другими артистами, внешние ссылки и альбомы.

WikiApi обращается к двум другим серверам, поэтому использует полные адреса, которые Dio не склеивает с базовым. Метод getSitelinks запрашивает у Викиданных названия статей (действие wbgetentities, параметр origin=* нужен, чтобы Викиданные отдали заголовок CORS), метод getSummary — первый абзац статьи из REST-интерфейса Википедии, метод getImages — фотографии исполнителей запросом SPARQL к query.wikidata.org.

3.4 Источник данных и ошибки

NetworkArtistDataSource составляет запрос Lucene. Каталог — исполнители и группы с тегами «hip hop» и «rap»: tag:"hip hop" AND tag:rap AND (type:person OR type:group). При поиске к нему добавляется условие на имя: каждое слово запроса пользователя экранируется от служебных символов Lucene и становится началом слова в имени, например «lil way» превращается в artist:(lil* AND way*).

Каждый вызов API-клиента обёрнут в метод _guard, который ловит DioException и бросает LoadException. Соответствие ошибок приведено в таблице 3. Для запроса карточки исполнителя ответ 400 тоже считается отсутствием записи: так MusicBrainz отвечает на идентификатор не в формате MBID, например на ссылку с опечаткой.

Таблица 3 — Перевод DioException в LoadException

Тип DioException	LoadException	Что видит пользователь
connectionTimeout, sendTimeout, receiveTimeout, transformTimeout, connectionError, badCertificate, cancel, unknown	ConnectionLoadException	Не удалось связаться с сервером, проверьте подключение к сети
badResponse с кодом 404 (у карточки также 400)	NotFoundLoadException	Такой записи в каталоге нет
badResponse с кодом 503	ServerLoadException(503)	Сервер просит подождать: слишком много запросов
badResponse с другим кодом	ServerLoadException(код)	Сервер ответил ошибкой

LoadException объявлен как sealed-класс в common/network, поэтому выражение switch на экране обязано перечислить все варианты: новая разновидность ошибки без подписи не скомпилируется.

3.5 Маппер и репозиторий

Расширение ArtistDtoMapper на ArtistDto переводит ответ сервера в ArtistModel. Теги сортируются по числу голосов; пять первых становятся тегами карточки, а те, что совпадают с жанрами каталога (hip hop, east coast hip hop, boom bap, trap и другие — всего 25), становятся списком жанров. Из псевдонимов убираются подсказки для поиска (тип Search hint) и повторы имени, из релиз-групп остаются номерные альбомы без вторичных типов, то есть без сборников, микстейпов и концертных записей. Связанные артисты берутся из связей типов «участник группы», «подгруппа», «совместный проект» и «настоящее имя», без повторов, не более восьми.

Репозиторий в методе getArtist сначала получает карточку из MusicBrainz, а затем по ссылке на Викиданные загружает справку. Английская и русская статьи грузятся параллельно через Future.wait. Выбор Future.wait вместо записи (a, b).wait сделан намеренно: при ошибке пара бросает ParallelWaitError, внутри которой исходную ошибку нужно искать, а Future.wait бросает первую исходную ошибку как есть.

Ошибка NotFoundLoadException в репозитории превращается в null, и Cubit показывает «Такой записи в каталоге нет». Остальные ошибки MusicBrainz уходят наверх без изменений. Единственное место, где ошибка

не поднимается выше, — справка из Википедии: это дополнение с другого сервера, и если оно не загрузилось, деталь всё равно показывается полностью, только без раздела «Об исполнителе». Это решение записано в коде комментарием, чтобы его не приняли за проглоченную ошибку.

3.6 Фотографии исполнителей

Фотографии загружаются одним запросом SPARQL на всю страницу списка: в запрос передаются MBID тридцати исполнителей страницы, а Викиданные возвращают пары «MBID — файл на Викискладе». Для детали в тот же запрос попадают исполнитель и все его связанные артисты, поэтому у связанных тоже есть снимки. Запрос идёт на другой сервер и ограничение MusicBrainz не расходует; на детали он отправляется одновременно с загрузкой справки.

Викиданные отдают изображение адресом вида Special:FilePath/<имя файла>. Этот адрес отвечает цепочкой редиректов без заголовка CORS, и в браузере такая картинка не загружается: Flutter в вебе рисует изображения через WebGL, поэтому браузер проверяет CORS у каждого ответа цепочки. Поэтому маппер WikidataImagesMapper сам строит прямой адрес миниатюры шириной 330 пикселей на upload.wikimedia.org: путь складывается из первых символов MD5 от имени файла (пакет crypto), а этот сервер заголовок CORS отдаёт.

Виджет ArtistArtwork выбирает снимок по порядку: локальный файл из ресурсов (для 33 исполнителей из второй работы, с указанием автора в docs/CREDITS.md), снимок из сети, буквенная плитка. Пока снимок из сети грузится или если он не загрузился, показывается плитка. Как и справка, фото — дополнение: у запроса SPARQL отдельный таймаут 5 секунд, и при его сбое список показывается с плитками, а не ошибкой. На живых данных фото нашлось у 27–29 исполнителей из 30 на странице; у остальных, например у The Notorious B.I.G., свободного снимка в Викиданных нет.

3.7 Состояния и Cubit

Состояние списка осталось одним закрытым типом, к нему добавились ошибка и подгрузка: Loading(query) | Loaded(artists, query, total, nextOffset, nextPage) | NotFound(query) | Error(query, error). Поле nextPage — перечисление idle | loading | error для подгрузки под уже показанным списком:

ошибка следующей страницы не стирает загруженные записи, а показывает под ними сообщение и кнопку «Повторить».

Поиск отправляется через 400 миллисекунд после последнего нажатия клавиши: без этой задержки ввод слова из шести букв дал бы шесть запросов, которые при ограничении сервера выстроились бы в очередь на шесть секунд. Защита от устаревшего ответа из второй работы сохранена: Cubit помнит последний запрос и отбрасывает ответы на предыдущие. После каждого await проверяется isClosed, поэтому ответ, пришедший после закрытия экрана, не вызывает ошибку «Cannot emit new states after calling close».

Метод loadMore просит следующую страницу со смещения nextOffset. Смещение хранится отдельно от длины списка, потому что записи, уже показанные на предыдущих страницах, отбрасываются по идентификатору: выдача поиска MusicBrainz упорядочена по релевантности и между запросами может сдвигаться. Если сервер вернул пустую страницу, общее число записей приравнивается к смещению, и подгрузка прекращается.

Состояние детали дополнено вариантом Error(error), кнопка «Повторить» вызывает загрузку того же идентификатора заново.

3.8 Экраны

Экраны изменились в части загрузки и ошибки. Общий виджет ErrorView в common/widgets показывает сообщение и кнопку «Повторить», текст сообщения по виду ошибки выбирает расширение LoadExceptionX из presentation/utils. В ARB-файлы добавлены пять ключей на обоих языках.

Сетка списка обёрнута в NotificationListener: когда до конца прокрутки остаётся меньше высоты экрана, вызывается loadMore. Повторный вызов во время загрузки Cubit игнорирует. Под сеткой показывается индикатор подгрузки или сообщение об ошибке следующей страницы, а счётчик над сеткой берётся из общего числа записей на сервере, а не из числа загруженных.

4 ПРОВЕРКА РАБОТЫ

Статический анализ flutter analyze замечаний не выдаёт, проверка паритета ключей tool/check_arb.dart подтверждает 55 ключей в двух ARB-файлах, сборка flutter build web выполняется. Файл с моками удалён, в каталоге lib моковых данных нет.

Цепочка слоя данных отдельно проверялась на живом сервере: первая и вторая страницы каталога по 30 записей из 4465 без пересечений, поиск «kendr» находит Kendrick Lamar, поиск несуществующего имени даёт ноль записей, карточка Eminem приходит с жанрами, псевдонимами, альбомами, связанными артистами и справкой на двух языках, запрос несуществующего идентификатора даёт null, а неверный адрес сервера — ConnectionLoadException.

Работа приложения проверялась в браузере Google Chrome. Отсутствие сети имитировалось блокировкой запросов к musicbrainz.org в браузере, ответ 503 — подменой первого ответа сервера: после подмены приложение само повторило запрос и показало список. Настоящий ответ 503 от MusicBrainz также попал на снимок вкладки Network: за ним сразу идёт автоматический повтор с ответом 200. Сценарий «открыть деталь и сразу вернуться назад» ошибок в консоль не выводит.

Загрузка и список показаны на рисунках 1–2, загрузка и содержимое детали — на рисунках 3–5, ошибка и повтор — на рисунках 6–7, поиск — на рисунках 8–9, подгрузка следующей страницы — на рисунке 10. Запросы к серверам на вкладке Network инструментов разработчика приведены на рисунках 11–13.

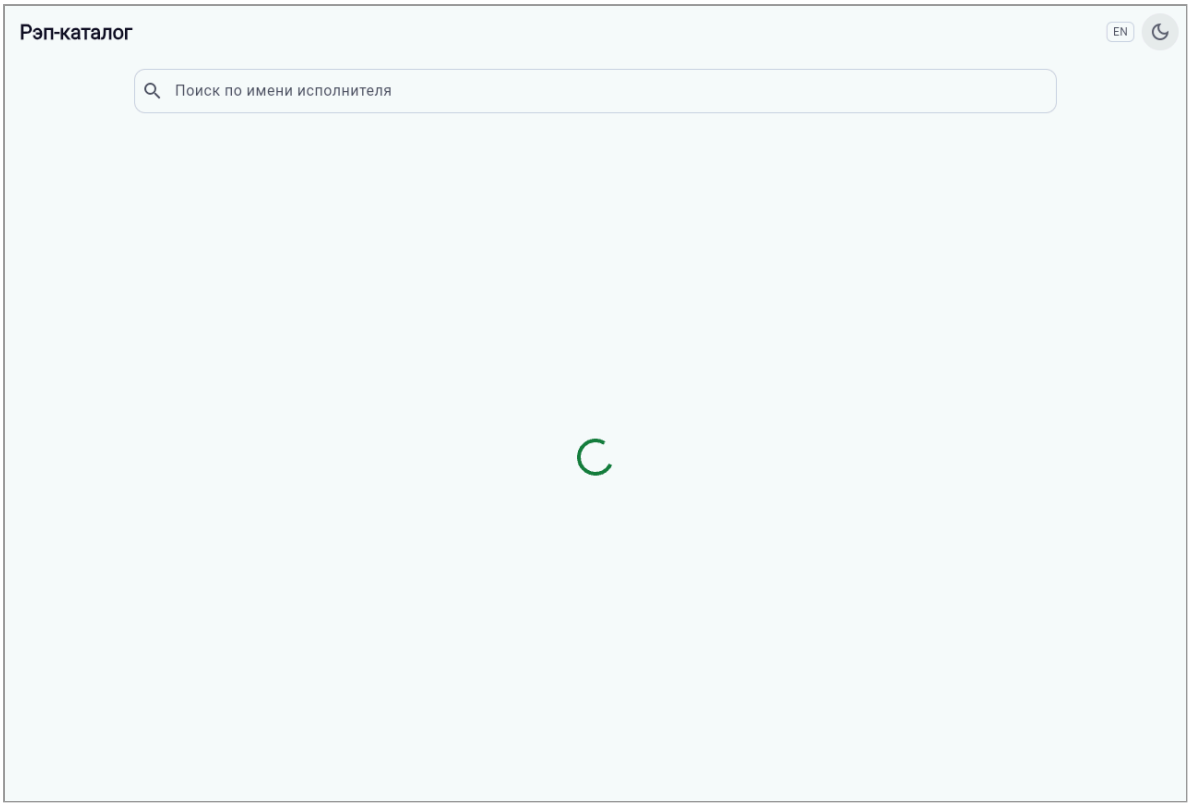


Рисунок 1 — Индикатор загрузки списка

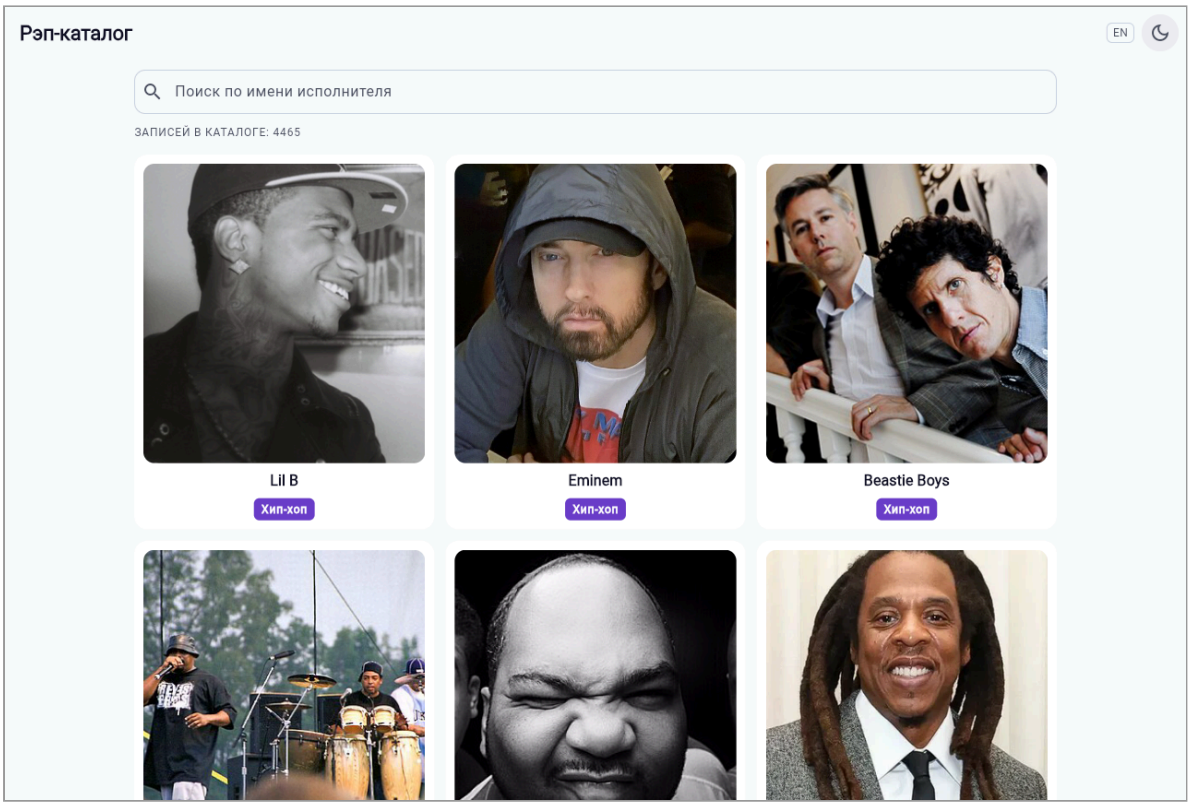


Рисунок 2 — Список из сети: 4465 записей в каталоге

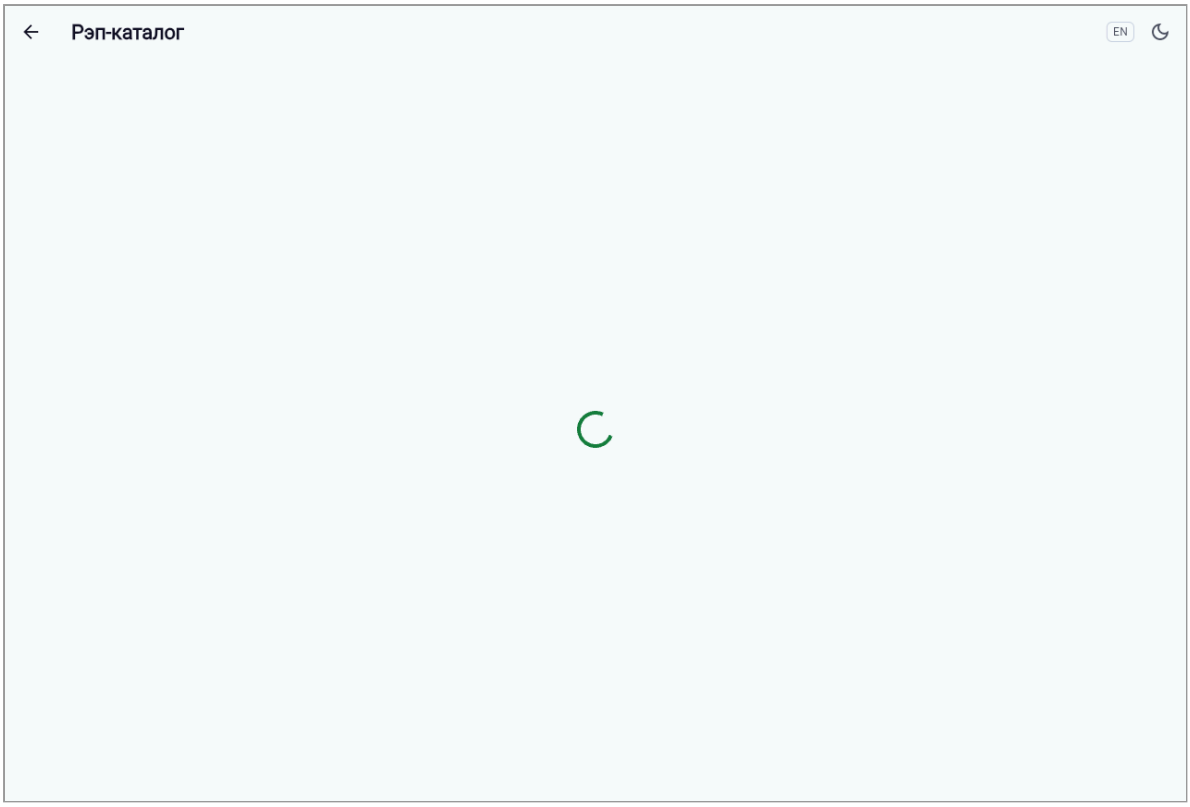


Рисунок 3 — Индикатор загрузки детали

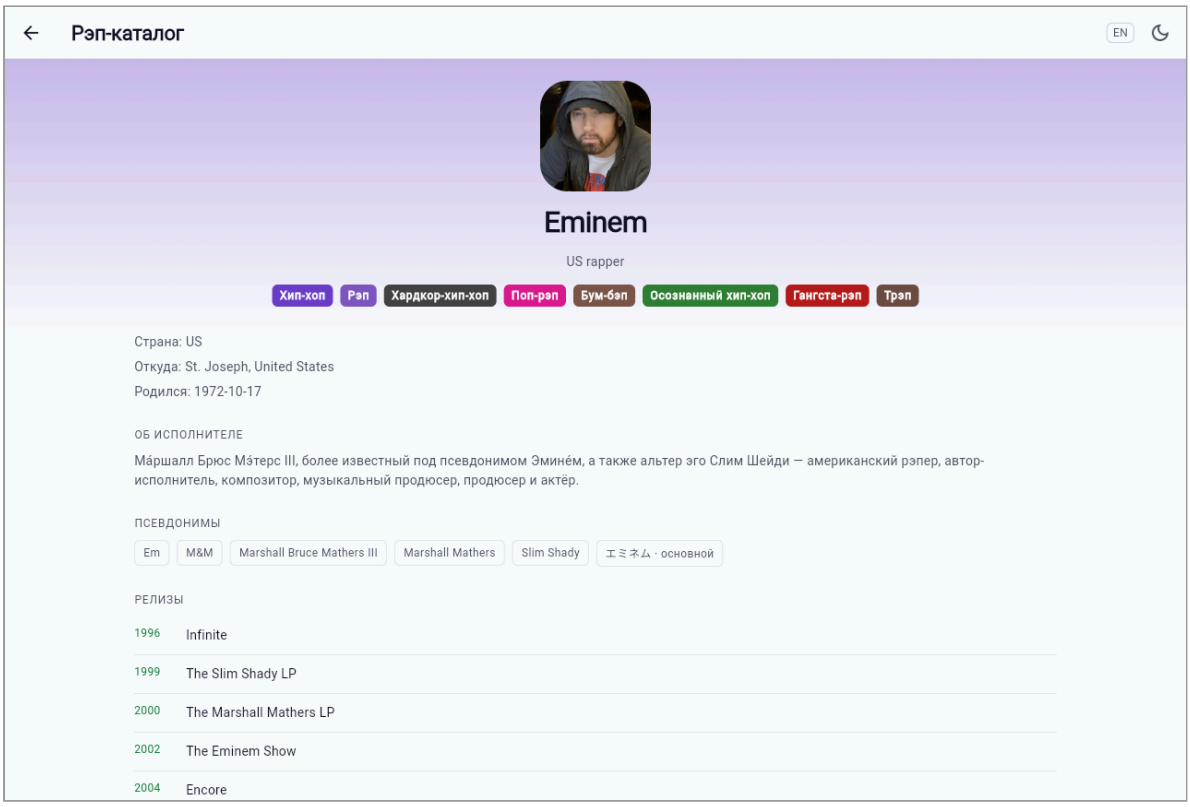


Рисунок 4 — Деталь артиста: факты, справка, псевдонимы, альбомы

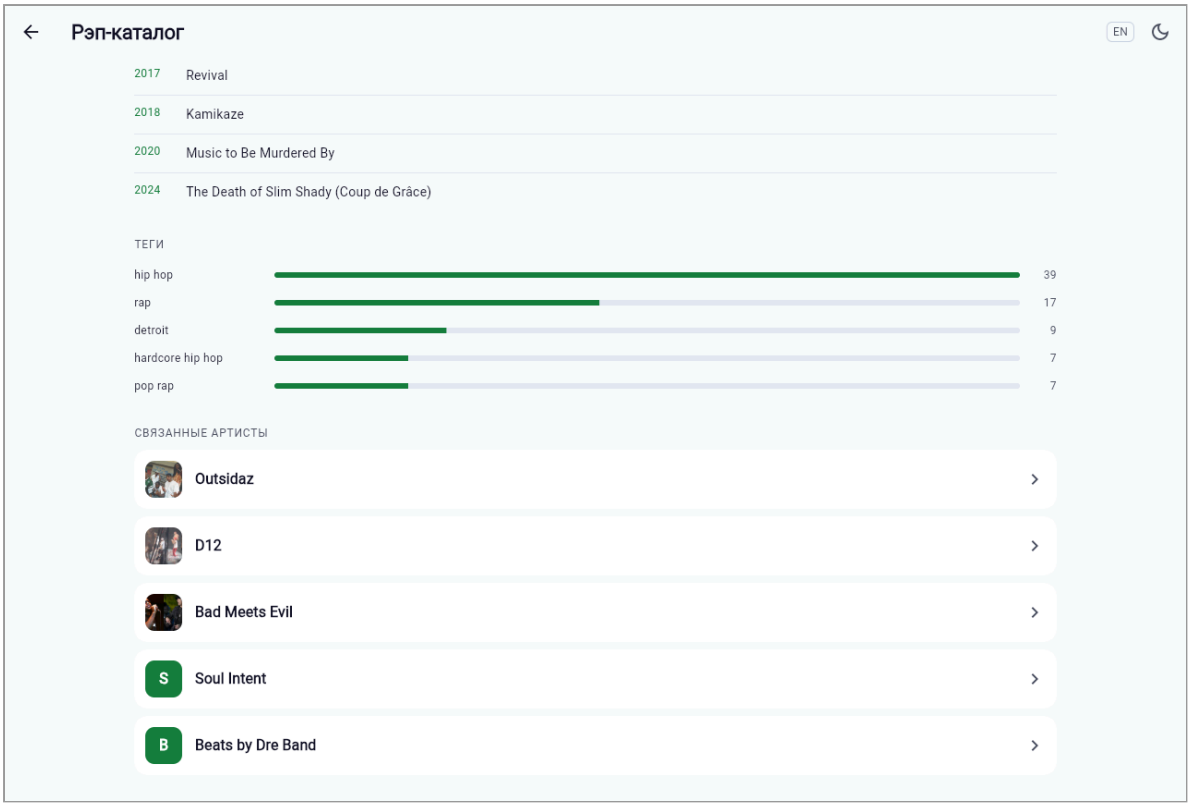


Рисунок 5 — Деталь артиста: теги и связанные артисты

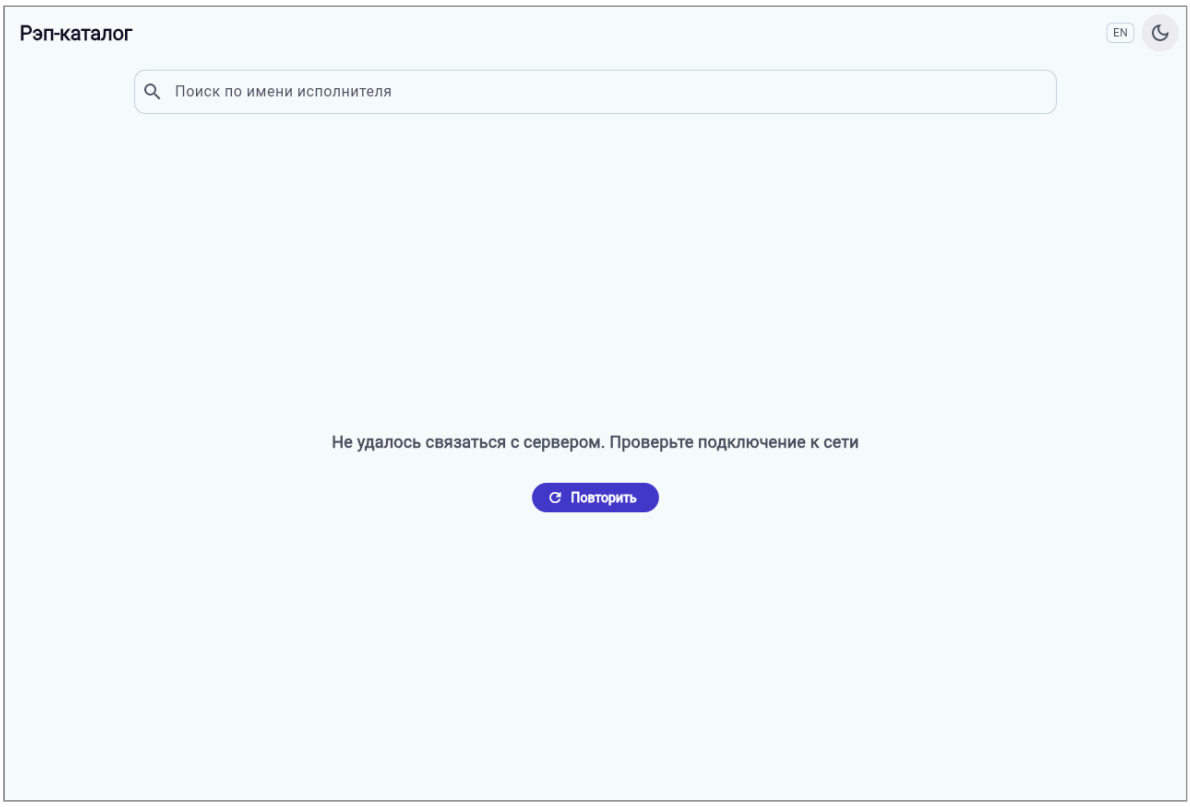


Рисунок 6 — Нет связи с сервером: сообщение и кнопка «Повторить»

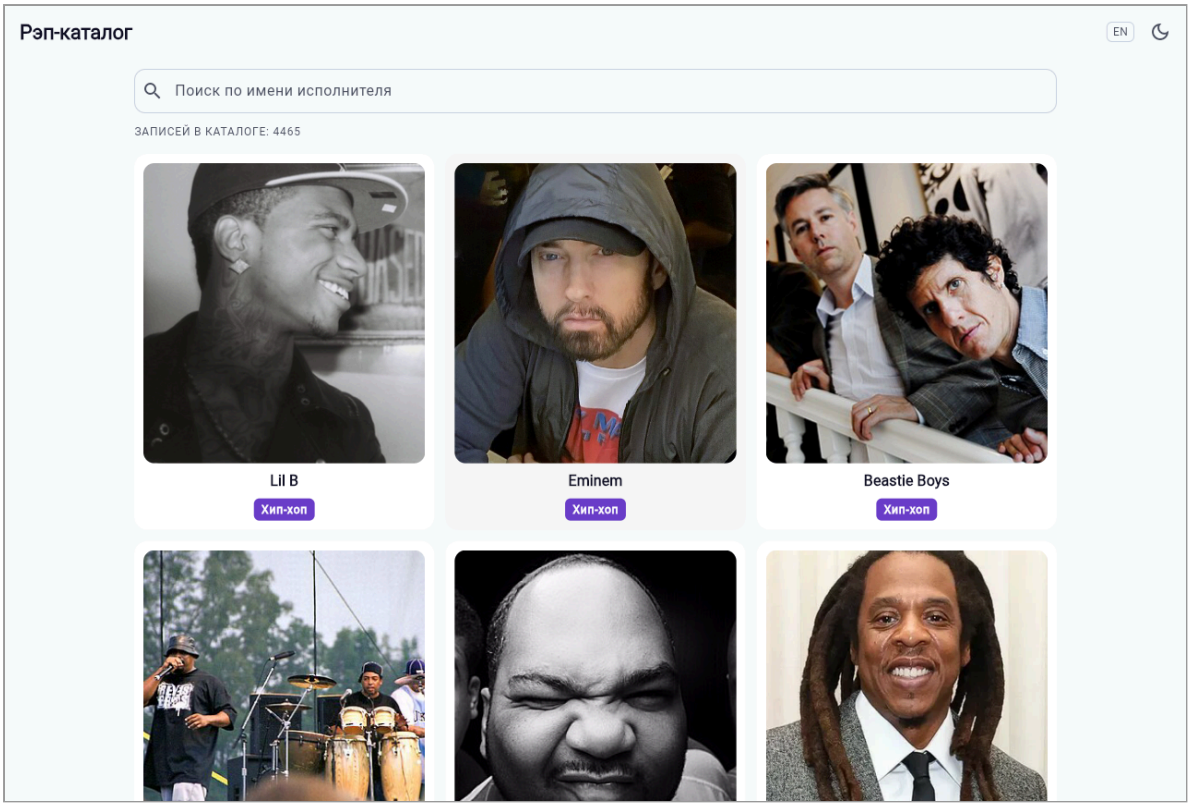


Рисунок 7 — После восстановления сети и нажатия «Повторить» список загружен

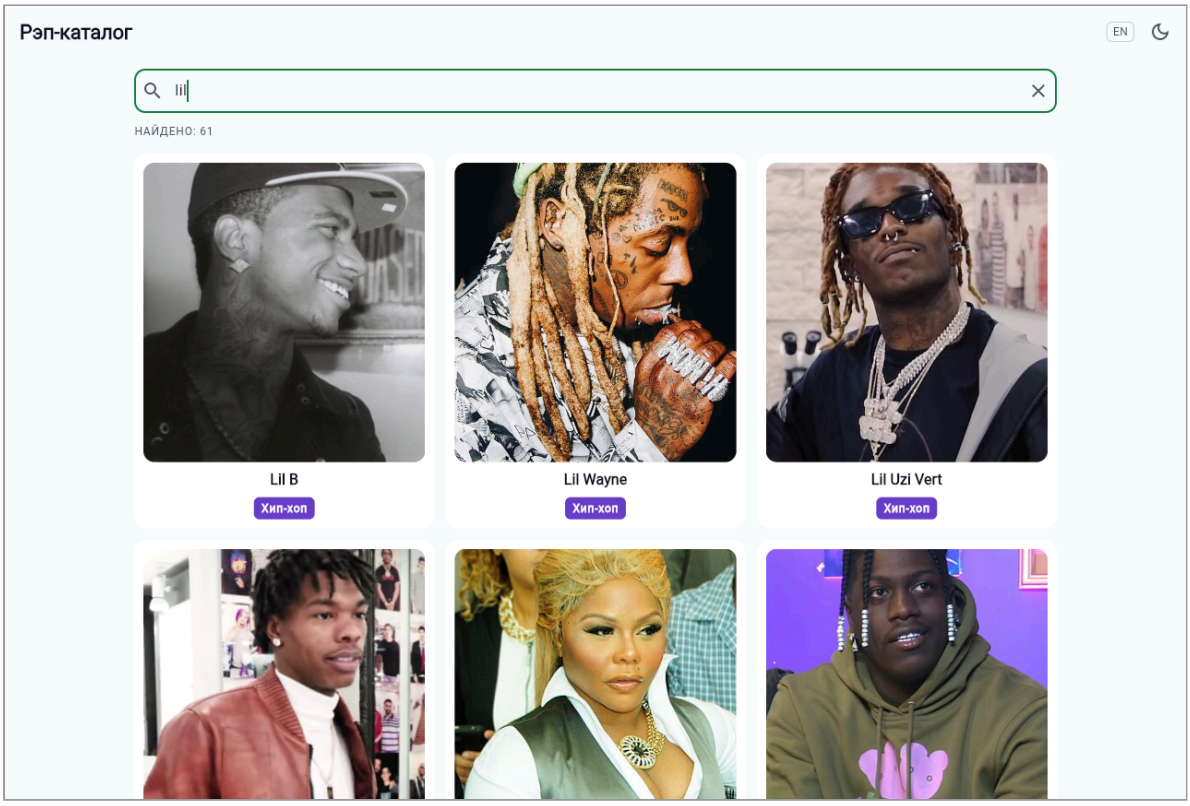


Рисунок 8 — Поиск по началу имени «lil»

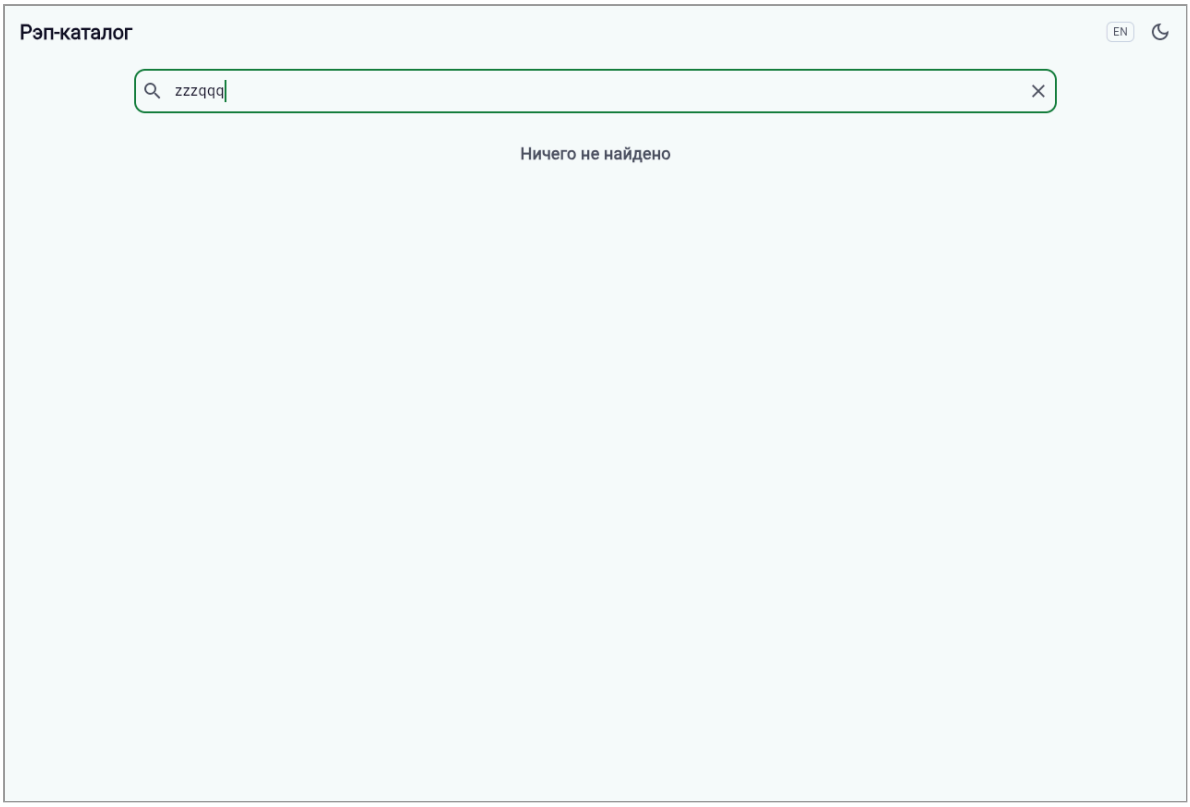


Рисунок 9 — Поиск без совпадений

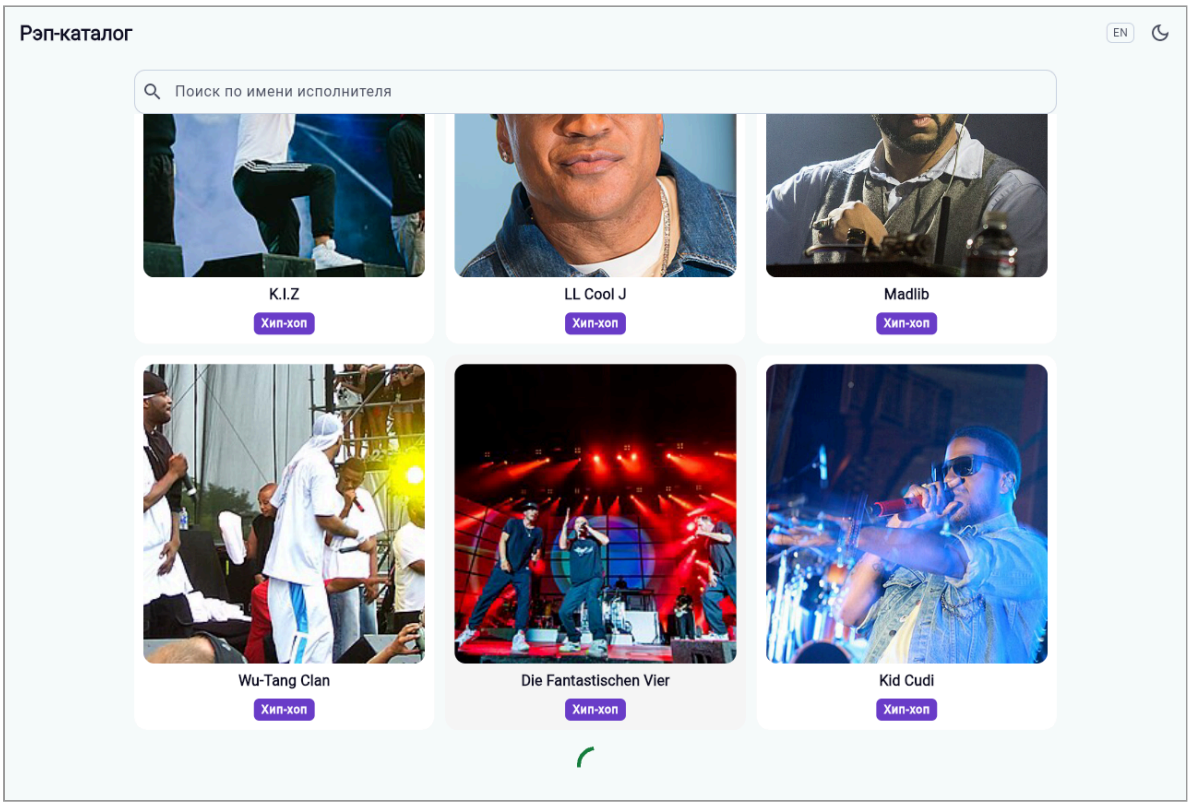


Рисунок 10 — Подгрузка следующей страницы при прокрутке

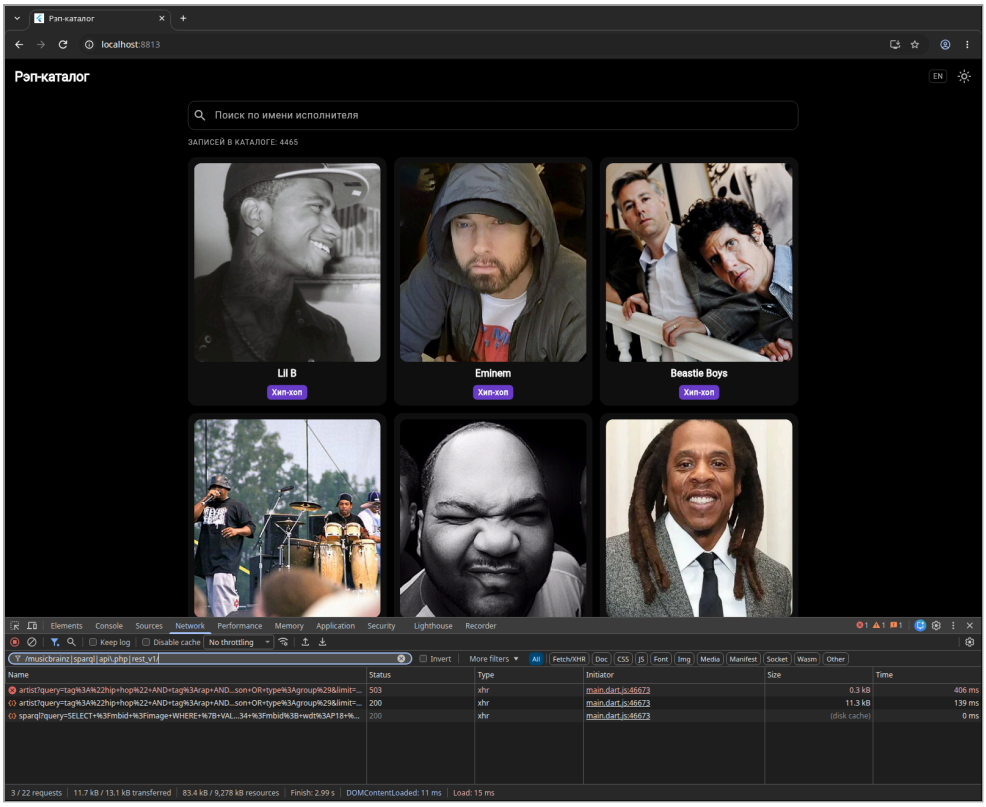


Рисунок 11 — Вкладка Network: ответ 503 от MusicBrainz, автоматический повтор и запрос фото к Викиданным

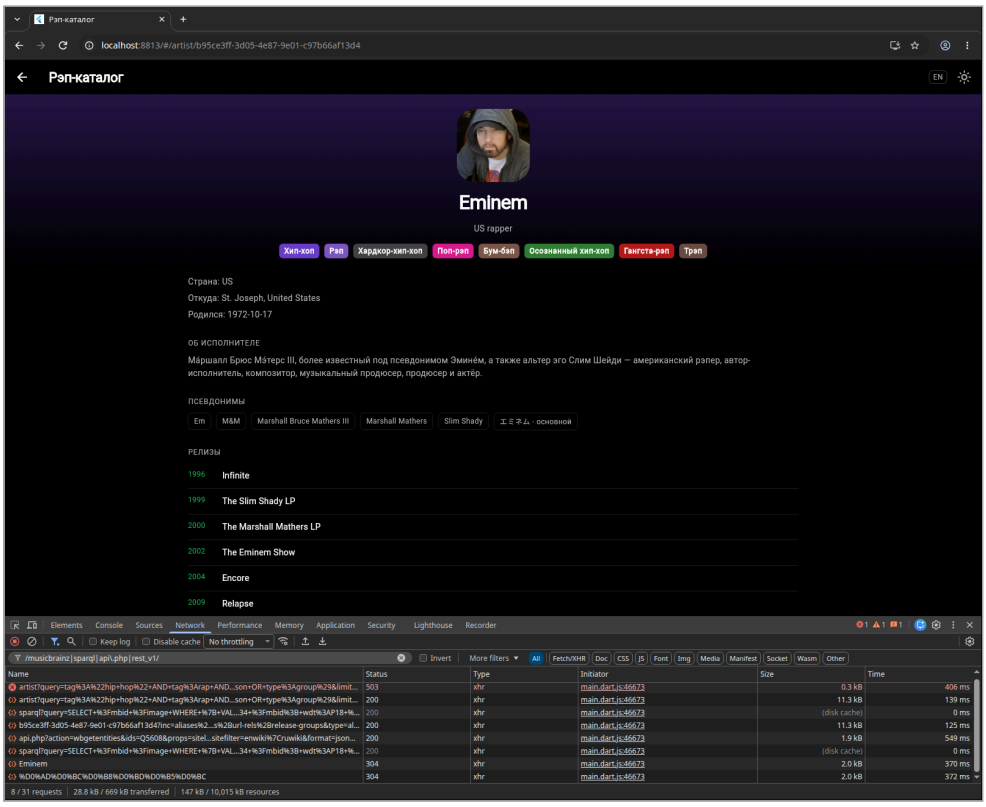


Рисунок 12 — Вкладка Network: карточка из MusicBrainz, фото и статьи из Викиданных, две статьи Википедии

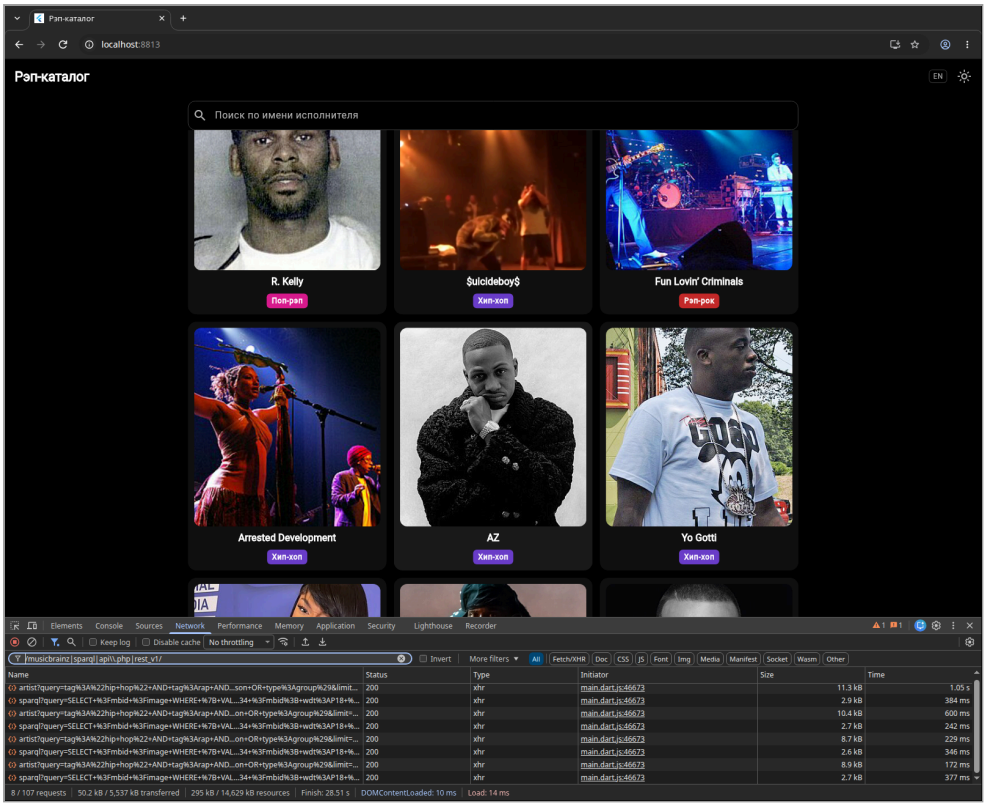


Рисунок 13 — Вкладка Network: страницы списка и их фото при прокрутке

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы поставленная цель достигнута: моковые данные приложения «Рэп-каталог» заменены данными программного интерфейса MusicBrainz. Все поставленные задачи выполнены.

Слой данных построен цепочкой «API-клиент — источник данных — репозиторий»: API-клиенты возвращают DTO, сгенерированные json_serializable, источник данных переводит DioException в закрытую иерархию LoadException, репозиторий через маппер отдаёт доменные модели. Dio создаётся один раз в корне приложения, адрес сервера записан в одном месте, Cubit и экраны не знают ни о dio, ни о DTO.

Экраны показывают загрузку и ошибку, кнопка «Повторить» возвращает данные после восстановления сети, поиск без совпадений по-прежнему выводит «Ничего не найдено», а не маскирует сетевую ошибку. Список подгружается постранично без повторов, справка об исполнителе на двух языках загружается параллельно, а фотографии всех исполнителей приходят из Викиданных одним запросом на страницу.

Отдельный вывод сделан по ограничениям источника. Лимит в один запрос в секунду повлиял на устройство приложения сильнее, чем ожидалось: он потребовал очереди запросов, задержки поиска при вводе, повтора при ответе 503 и отказа от отдельных запросов за связанными артистами. Практическая ценность работы состоит в полученном опыте построения слоя данных, обработки ошибок сети и согласования приложения с ограничениями внешнего сервера. Следующая веха курса — кэширование и сохранение данных на диск.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fetch data from the internet : документация Flutter [Электронный ресурс]. — URL: https://docs.flutter.dev/cookbook/networking/fetch-data (дата обращения: 07.11.2026).

2. JSON and serialization : документация Flutter [Электронный ресурс]. — URL: https://docs.flutter.dev/data-and-backend/serialization/json (дата обращения: 07.11.2026).

3. Guide to app architecture. Data layer : документация Flutter [Электронный ресурс]. — URL: https://docs.flutter.dev/app-architecture/guide (дата обращения: 07.11.2026).

4. Futures and error handling : документация Dart [Электронный ресурс]. — URL: https://dart.dev/libraries/async/futures-error-handling (дата обращения: 07.11.2026).

5. dio : пакет HTTP-клиента для Dart [Электронный ресурс]. — URL: https://pub.dev/packages/dio (дата обращения: 07.11.2026).

6. json_serializable : генератор кода сериализации JSON [Электронный ресурс]. — URL: https://pub.dev/packages/json_serializable (дата обращения: 07.11.2026).

7. Flutter Infinite List : руководство flutter_bloc [Электронный ресурс]. — URL: https://bloclibrary.dev/tutorials/flutter-infinite-list/ (дата обращения: 07.11.2026).

8. MusicBrainz API : описание программного интерфейса [Электронный ресурс]. — URL: https://musicbrainz.org/doc/MusicBrainz_API (дата обращения: 07.11.2026).

9. MusicBrainz API. Rate Limiting [Электронный ресурс]. — URL: https://musicbrainz.org/doc/MusicBrainz_API/Rate_Limiting (дата обращения: 07.11.2026).

10. Wikibase API : wbgetentities [Электронный ресурс]. — URL: https://www.wikidata.org/w/api.php?action=help&modules=wbgetentities (дата обращения: 07.11.2026).

11. Wikidata Query Service : руководство [Электронный ресурс]. — URL: https://www.wikidata.org/wiki/Wikidata:SPARQL_query_service (дата обращения: 07.11.2026).

12. Wikimedia Commons : миниатюры и адреса файлов [Электронный ресурс]. — URL: https://www.mediawiki.org/wiki/Manual:$wgHashedUploadDirectory (дата обращения: 07.11.2026).

13. Wikimedia REST API : page summary [Электронный ресурс]. — URL: https://en.wikipedia.org/api/rest_v1/ (дата обращения: 07.11.2026).